

SCoPE: Parallel Decoding with Auction Refinement and Learned Repair for Dynamic Weapon-Target Assignment

Jaejin Lee · George Runger

Received: date / Accepted: date

Abstract The Dynamic Weapon-Target Assignment Problem (DWTAP) extends the classical weapon-target assignment problem with multiple engagement stages, weapon reload times, ammunition limits, target availability windows, and platform-level engagement compatibility. We propose SCoPE (Simultaneous COmmitment with Parallel rEpair): a heterogeneous weapon-target graph encoder feeding a parallel multi-pointer decoder, in which all weapons commit their actions simultaneously at every stage; a many-to-one auction that assigns targets to the firing weapons by greedy marginal value, with a provable per-stage approximation guarantee; a weapon-to-weapon self-attention (communication) layer that lets each weapon’s fire-or-hold decision, not just its target choice, condition on a learned summary of every other weapon’s state that stage; and a learned repair policy that revisits the completed schedule, flips individual fire-or-hold decisions, re-simulates, and keeps the best result, which by construction cannot degrade the schedule it is given and reduces the mean objective by a further 13% relative across all tested configurations. Trained only on small instances (5–7 weapons, targets, and stages) with reward-to-go REINFORCE, a shared parallel-rollout baseline, and no critic, SCoPE generalizes zero-shot to held-out instances as large as 70 weapons, 100 targets, and 15 stages. On resource-scarce instances – the realistic regime for this problem, where ammunition and weapon platforms are limited relative to the threats they face and holding fire for a better opportunity is a genuine trade-off – SCoPE reaches a mean 95.2% of the best-found solution’s destroyed value across twelve held-out configurations spanning this scale range, outperforming a classical marginal-return heuristic and the auction mechanism used in isolation at every configuration, outperforming standard neural combinatorial optimization baselines (AM, POMO) at all but one, and matching or closely approaching a fully sequential decoder with the same encoder and training

Jaejin Lee
Intel Corporation, Chandler, AZ 85226, USA
E-mail: jaejin.lee@intel.com

George Runger
School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ 85281, USA
E-mail: george.runger@asu.edu

recipe at a fraction of its per-instance decoding cost. Against an exact solver run under a fixed 600-second time budget, SCoPE overtakes it at the largest five of twelve configurations, where the solver’s branch-and-bound no longer finds a competitive incumbent in time; at smaller scales, where the solver still proves or nearly proves optimality, SCoPE remains within roughly 1–15% of that optimum.

Keywords Reinforcement learning · Multi-agent coordination · Combinatorial optimization · Dynamic weapon-target assignment · Graph neural networks

1 Introduction

Multi-agent construction methods for combinatorial optimization treat simultaneous decision-making as a coordination problem to be engineered away: when several agents commit at once, the standard response is to add a learned conflict handler or a communication protocol so that they do not collide [3, 53, 14]. On the dynamic weapon-target assignment problem (DWTAP), whether this is necessary turns out to depend on a property of the *instance*, not just the problem class: how scarce weapon resources are relative to the threats they face. Under resource-abundant instances – the regime conventionally used to benchmark WTA solvers – M weapons choosing their targets simultaneously, refined only by a marginal-value auction that resolves which target each already-firing weapon should engage, match a fully sequential decoder closely, with no learned coordination mechanism of any kind. Under resource-scarce instances – where ammunition and platforms are genuinely limited, and holding fire for a better opportunity later is a live option – the same architecture falls substantially behind sequential decoding instead. We show this is not a failure of the underlying hypothesis but an incomplete one: the auction coordinates *which* target a firing weapon should engage, not *whether* it should fire at all, and it is exactly that second, participation-level coordination that resource scarcity makes consequential. Diagnosing this gap instance by instance against exact-solver output, and closing it with a single added communication layer, is the subject of this paper.

The weapon-target assignment problem allocates weapons to targets to minimize the expected surviving target value [42], and is NP-hard even in its static form [38]. The dynamic variant studied here adds the features that make the problem operational: multiple engagement stages, weapon reload times, ammunition limits, target availability windows, and platform-level engagement compatibility (Q_m). These dynamics are not a modeling convenience – they impose a clock. Engagement windows open and close, weapons cycle through reload, and the value of an assignment can change before the previous decision has finished playing out, which is why receding-horizon treatments re-solve the assignment at every stage rather than committing to one static plan [64, 63]. A method whose per-decision cost grows into minutes at realistic sizes cannot keep pace with the cycle it serves, regardless of solution quality. Decision speed is therefore a constraint of the problem, not an implementation detail. Resource scarcity is likewise not an artificially hard test case but the operating condition the problem exists to model: a defender’s ammunition and platforms are, by the premise of the engagement, limited relative to the threats presented, and it is exactly that scarcity which makes the choice to engage now versus hold in reserve a real decision rather than a formality. Instances in which resources are comparatively abundant relative to targets – where holding fire is rarely the better option –

under-test a solution method’s ability to reason about that trade-off, even though they are the instances most commonly reported in prior WTA evaluation.

Existing approaches sit on the wrong side of that constraint in different ways. Exact methods certify optimality but saturate at small instances and cannot be re-invoked every stage at scale [1, 29]; heuristics are fast but tied to the constraint structure they were designed around [30, 37]; and prior reinforcement-learning approaches either address only the static problem, fix their network dimensions to the instance size, or – when they do scale – construct the assignment through a single sequential policy whose decoding cost grows with the number of weapons [46, 21, 59, 61], precisely the dependence a real-time setting cannot afford (Section 2, Table 1).

Our central claim is structural, and has two parts. First, whether simultaneous decoding needs a coordination mechanism at all depends on the *type of coupling* between agents, not the number of agents: in routing- and scheduling-style problems, agents contend for resources that admit one owner – coupling is *hard*, and a conflict handler is doing real work – while in DWTAP, multiple weapons may legally engage the same target, and what disciplines them is the multiplicative, diminishing-returns objective, under which piling onto an already-doomed target is merely unprofitable rather than infeasible. Coupling here is *soft*. Second, soft coupling is necessary but not sufficient for zero-communication parallel decoding to be safe: an auction that resolves target contention among already-firing weapons leaves the prior, participation-level decision – fire or hold – entirely uncoordinated, and under resource scarcity that omission is measurable. We show – theoretically (Section 6.5) and empirically (Section 7.5) – that restoring coordination at this level requires only a lightweight, single-pass communication mechanism, not a return to sequential decoding.

We instantiate this claim in SCoPE (Simultaneous COmmitment with Parallel rEpair): a heterogeneous weapon-target graph encoder feeding M simultaneous per-weapon pointer policies (Section 6.3), refined at inference by a many-to-one marginal-value auction restricted to deciding *which* target each already-firing weapon engages (Section 5), and augmented with a weapon-to-weapon self-attention layer that lets each weapon’s fire-or-hold decision, not just its target choice, condition on a learned summary of every other weapon’s state this stage (Section 6.2). Trained end-to-end with reward-to-go REINFORCE and a shared parallel-rollout baseline on small instances only (5–7 weapons, targets, and stages), the policy generalizes zero-shot to held-out instances an order of magnitude larger. The training recipe itself contains this paper’s third empirically-grounded finding: adding a learned critic with potential-based reward shaping – our original design – consistently *hurt* held-out generalization, and the final recipe uses no critic at all (Sections 6.6, 7.7).

The main contributions of this study are summarized as follows:

- **Realistic dynamic battlefield modeling under resource scarcity:** Extension of WTAP to dynamic settings with multiple time steps, weapon reload constraints, ammunition limits, target availability windows, and weapon-target engagement compatibility (Q_m), and an argument – supported empirically (Section 7.5) – that resource-scarce instances, not the resource-abundant instances conventionally reported, are the realistic and diagnostically informative regime for this problem.
- **A structural distinction for multi-agent combinatorial optimization, refined:** We characterize DWTAP as a *soft-coupled* multi-agent assignment problem, in which coordination among weapons arises from the nonlinear, multiplica-

tive structure of the objective (Eq. (1)) rather than from hard resource exclusivity as in routing- and scheduling-style multi-agent problems [3]. We show that soft coupling alone is sufficient to induce coordinated target engagement under resource abundance, but not under resource scarcity, where a further, participation-level coordination gap emerges that an auction-only refinement cannot close.

- **A minimal fix, diagnosed instance by instance:** Comparing SCoPE’s own decisions against exact-solver output instance by instance, we identify the specific failure mode (systematic under-use of available ammunition, not miscalibrated temporal judgment) and show that a single added weapon-to-weapon self-attention layer, evaluated at the same inference cost order as the conflict-handler-free decoder, closes most of the resulting gap to sequential decoding.
- **Rigorous, standardized benchmarking:** SCoPE is trained on a small-scale multi-scale curriculum but generalizes zero-shot to significantly larger held-out instances, evaluated across four size tiers up to $70 \times 100 \times 15$ under both resource-abundant and resource-scarce instance distributions. We evaluate against an exact solver (SCIP), a classical marginal-return heuristic (Greedy), the auction refinement in isolation, and standard neural combinatorial optimization baselines (AM [31], POMO [33]) trained under an identical protocol.

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 presents the problem formulation, Section 4 details the MDP formulation, Section 5 introduces the per-stage auction assignment as a preliminary, Section 6 describes the proposed SCoPE methodology, Section 7 reports experimental results, and Section 8 concludes with insights and future directions.

2 Related Work

We review four strands: exact and heuristic WTA methods, reinforcement learning for WTA (Table 1), multi-agent neural construction, and the inference-time techniques we adopt as tools.

2.1 Exact and Heuristic Approaches

The WTAP, introduced by Manne [42] and NP-hard even in its static form [38], was first made tractable through linearization or identical-weapon assumptions [15, 19, 24, 9, 28]. Exact methods now certify optimality on moderate static instances – up to 80×80 via specialized branch-and-bound [1] – but degrade sharply beyond that scale [27, 29], while heuristics and metaheuristics [30, 35, 36, 37, 17, 52, 11, 45] are fast but bound to the constraint structure they were designed around. Neither meets the dynamic setting’s demand (Section 1) for fast, repeatable decisions at scale without per-variant redesign.

2.2 Reinforcement Learning for WTA

Early neural formulations date to Wacholder [57]; Table 1 positions the reinforcement-learning methods that followed along the axes that matter here. Static-WTA meth-

ods either tie network dimensions to the instance size [44, 51, 21] or achieve size-agnostic architectures at the cost of omitted operational constraints or identical-weapon assumptions [40, 46, 56]. Dynamic methods each inherit one of the same limits: size-fixed networks [59], simplified identical-agent objectives without WTA’s nonlinearity [26], or receding-horizon decomposition with no learned cross-stage policy [64, 63]. Closest to us, Yoon, Lee, and Cho [61] target scalable DWTA with a transformer-based policy; SCoPE differs in decoding all weapons’ actions in parallel rather than through a single sequential policy, characterizing when that is safe (Section 6.5), and modeling platform-level engagement compatibility Q_m . Most recently, Oh et al. [48] also pair graph neural networks with reinforcement learning for DWTA, training a graph-attention policy with PPO under a partially observable formulation and evaluating on naval and ground scenarios against heuristic and metaheuristic baselines; their contribution is scenario realism through the OODA decision cycle, whereas ours is the structural split of the per-stage decision into a learned participation policy and an assignment mechanism with a per-stage guarantee, benchmarked additionally against an exact solver and neural construction baselines. Pairing a learned decision with a classical combinatorial subroutine in this way follows [16]; [54] is the reverse, using reinforcement learning to learn an auction’s bidding values themselves.

Table 1 Reinforcement-learning approaches to weapon-target assignment along the axes that position this work.

Method	Dynamic	Size-agn.	Q_m	Parallel dec.
Meng et al. [44]	–	–	–	–
Shin et al. [51]	–	–	–	–
Luo et al. [40]	–	✓	–	–
Na & Choi [46]	–	✓	–	–
Gaudet et al. [21]	–	–	–	–
Wu et al. [59]	✓	–	–	–
Hu et al. [26]	✓	✓	–	–
Yoon et al. [61]	✓	✓	–	–
SCoPE (ours)	✓	✓	✓	✓

2.3 Multi-Agent Construction: Hard vs. Soft Coupling

Simultaneous multi-agent action selection builds on learned inter-agent communication – CommNet’s broadcast averaging [53], TarMAC’s attention-based addressing [14] – combined with pointer-style output heads [56], applied to scheduling by ScheduleNet [49]. PARCO [3] extends this lineage with a learned, priority-based conflict handler for *hard-coupled* problems, where simultaneous access to a resource is infeasible by construction. As argued in Section 1, DWTAP’s coupling is *soft* – contention is unprofitable under the multiplicative objective (Eq. (1)), not infeasible – so we build on the communication-and-parallel-pointer lineage directly, with no conflict handler, and use PARCO as the hard-coupled point of contrast. The cost of forgoing within-round communication is quantified empirically in Section 7.5 and characterized theoretically in Section 6.5.

2.4 Improving a Solution After It Is Built

Most neural construction work that improves on a policy at test time does so *during* construction: rollout algorithms [7] score candidate next actions by simulating forward under a base policy, simulation-guided beam search [13] keeps a width-limited frontier of partial solutions, and efficient active search [25] adapts policy parameters per instance. All three expand a partial solution and are therefore priced against the branching factor at each decision point – which for a parallel decoder is the joint action over M weapons, not a single edge.

A separate line instead learns to *improve a complete solution*, repeatedly selecting a local modification and accepting it if it helps: neural local rewriting [12], learning-based improvement heuristics for routing [60], and dual-aspect representations for the same [41]. Closest in shape to our pipeline, Zhang, Hu, and Zhao [62] construct a preliminary solution for the vehicle routing problem with time windows using an attention-based policy and then improve it with a large neighborhood search; the difference is the improvement operator, which in their case is a classical, hand-designed destroy-and-repair search, whereas ours is a second learned policy whose edits are accepted by re-simulation. This family has not, to our knowledge, been applied to DWTAP, and it fits the problem’s structure unusually well for two reasons developed in Section 6.4: the quality of a stage- t engagement often only becomes observable after later stages resolve, and re-simulating an edited schedule restores feasibility automatically rather than requiring a repair operator of its own. Section 6.4 adopts this paradigm for the participation decision specifically.

MatNet [34] encodes asymmetric compatibility matrices directly, a natural fit for DWTAP’s own asymmetric $P_{m,n}/Q_m$ structure, though it is not among the baselines evaluated in Section 7.4 (which uses AM and POMO); LEHD [39] exemplifies the train-small/test-large protocol our own evaluation follows (Section 7.1). We report all results over multiple independent seeds, in the spirit of robustness-oriented evaluation [23], and build our benchmark on RL4CO [4], which provides reference implementations of AM [31], POMO [33], and related baselines; recent surveys map the broader subfield [43, 8].

3 Problem Formulation

Unlike static WTAP, the proposed Dynamic Weapon-Target Assignment Problem (DWTAP) is structured across discretized time steps, denoted as $t = 1, 2, \dots, T$. In each time step t , at most one round of ammunition from each weapon m can be assigned to a target n . This discretization enables the model to incorporate the dynamics of the battlefield and the availability of resources at each specific interval, thereby facilitating the sequential decision-making process. Within this framework, we consider realistic combat scenario constraints, such as weapon reload time, ammunition limitation, and target time windows.

The objective is to minimize the total remaining target values, each denoted as V_n , by optimally assigning a set of weapons, denoted as M (where $m = 1, 2, \dots, M$), to a set of targets represented by N (where $n = 1, 2, \dots, N$). The assignment must adhere to the following constraints:

- Each weapon m can assign at most one round of ammunition to one target n at each time step t , but multiple rounds from different weapons can be assigned to a single target simultaneously.
- Each weapon, denoted as m , has a specified reload time D_m . After engaging a target, weapon m must wait D_m time units before it can be used again.
- Each weapon m has a limited amount of ammunition, represented by A_m , which must last across all time steps. Therefore, at any time t , weapon m can assign ammunition only if it has not depleted its ammunition A_m .
- Each target n has a specific vulnerability window, from $t_{n,\text{start}}$ to $t_{n,\text{end}}$. A target can only be engaged during this time frame.
- Each weapon m has an engagement compatibility set $Q_m \subseteq N$, reflecting platform-level restrictions such as range, guidance mode, and target-type suitability; weapon m may only be assigned to targets $n \in Q_m$.
- The proportion of target n 's value reduction when hit by weapon m , denoted by $P_{m,n}$, quantifies how effectively weapon m reduces the value V_n of target n .

3.1 Notation

Table 2 Notations for the Mathematical Model

Sets	
M	Set of weapons, indexed by $m = 1, \dots, M$
N	Set of targets, indexed by $n = 1, \dots, N$
T	Set of time stages, indexed by $t = 1, \dots, T$
Parameters	
V_n	Initial value of target n
D_m	Reload time for weapon m
A_m	Total ammunition for weapon m
TW_n	Time window for target n
Q_m	Engagement compatibility set for weapon m ($Q_m \subseteq N$)
$P_{m,n}$	Value reduction proportion
Decision Variables	
$x_{m,n,t}$	Binary assignment variable
$w_{m,t}$	Remaining readiness time
$a_{m,t}$	Remaining ammunition

3.2 Optimization Model

The problem can be formulated as a nonlinear integer programming problem. The objective function is:

$$\min \sum_{n=1}^N V_n \prod_{t=1}^T \prod_{m=1}^M (1 - P_{m,n})^{x_{m,n,t}} \quad (1)$$

Subject to constraints:

$$\sum_{n=1}^N x_{m,n,t} \leq 1, \quad \forall m, t \quad (2)$$

$$w_{m,t+1} = \max(0, w_{m,t} - 1) + D_m \sum_{n=1}^N x_{m,n,t}, \quad \forall m, t \quad (3)$$

$$x_{m,n,t} \leq 1 - \frac{w_{m,t}}{D_m}, \quad \forall m, n, t \quad (4)$$

$$a_{m,t+1} = a_{m,t} - \sum_{n=1}^N x_{m,n,t}, \quad \forall m, t \quad (5)$$

$$a_{m,t} \geq x_{m,n,t}, \quad \forall m, n, t \quad (6)$$

$$x_{m,n,t} = 0, \quad \text{if } t \notin TW_n, \quad \forall m, n \quad (7)$$

$$x_{m,n,t} = 0, \quad \text{if } n \notin Q_m, \quad \forall m, n, t \quad (8)$$

$$x_{m,n,t} \in \{0, 1\}, \quad \forall m, n, t \quad (9)$$

$$w_{m,t} \in \mathbb{Z}_+, \quad \forall m, t \quad (10)$$

$$0 \leq w_{m,t} \leq D_m, \quad \forall m, t \quad (11)$$

$$w_{m,1} = 0, \quad \forall m \quad (12)$$

$$a_{m,t} \in \mathbb{Z}_+, \quad \forall m, t \quad (13)$$

$$a_{m,0} = A_m, \quad \forall m \quad (14)$$

4 Markov Decision Process Formulation

The DWTAP can be modeled as a sequential decision-making problem, where a weapon is assigned to a target at each time step with the goal of minimizing the total remaining target values. This sequential decision-making process can be effectively modeled as a Markov Decision Process (MDP) [2].

An MDP, $\mathcal{M}$, is represented as a 5-tuple:

$$\mathcal{M} = (S, A, P, R, \gamma) \quad (15)$$

where S represents states, A denotes actions, P is the transition probability, R is the reward function, and γ is the discount factor.

The proposed DWTAP MDP is defined as follows:

- **State** (s_t): The state at stage t collects, for every compatible (weapon,target) pair, the nine quantities below; Section 6.1 shows how these same quantities factor into weapon-node, target-node, and edge features of a heterogeneous graph rather than being treated as $M \times N$ independent flat vectors:

$$s_t = [u_t; v_t; d_t; r_t; a_t; f_t; ws; we; p] \in \mathbb{R}^9 \quad (16)$$

where:

- u_t : Weapon readiness indicator
- v_t : Remaining target value
- d_t : Remaining reload time
- r_t : Remaining time to end
- a_t : Remaining ammunition
- f_t : Cumulative hits on target
- ws : Earliest attack time
- we : Latest attack time
- p : Value reduction proportion
- **Action** (a_t): One decision epoch corresponds to one time stage t of Section 3, and the action is the *joint* per-weapon assignment

$$a_t = (a_{1,t}, \dots, a_{M,t}), \quad a_{m,t} \in Q_m \cup \{\text{no-op}\}, \quad (17)$$

i.e. every weapon simultaneously selects one compatible target or holds fire, subject to the feasibility constraints of Section 3 (readiness, ammunition, time windows). This joint-action formulation is what the parallel decoder of Section 6.3 realizes directly; a sequential decoder factorizes the same joint action into M consecutive single-weapon sub-decisions within the stage.

- **Transition**: Deterministic with $P(s_{t+1}|s_t, a_t) = 1$.
- **Reward**:

$$R(s_{t+1}|s_t, a_t) = \frac{C(s_t) - C(s_{t+1})}{C(s_0)} \quad (18)$$

where $C(s_t)$ is the sum of target values at state s_t .

5 Preliminary: Per-Stage Target Assignment by Auction

The classical assignment problem [32] allocates a set of resources to a set of tasks to maximize total utility. This problem is solvable to optimality in polynomial time under the condition that the objective is linear in the individual assignments [10]. Bertsekas [5] proposed the original auction algorithm, an exact algorithm, for the one-to-one assignment problem, and it was later extended to many-to-one assignments such as the transportation problem [6].

The stage problem of DWTA is a many-to-one assignment with a nonlinear objective. Fix stage t , and suppose the set of firing weapons $F \subseteq \{1, \dots, M\}$ is given. Let $\rho_n^{(t)} \in [0, 1]$ be target n 's surviving value fraction entering the stage, so $V_n \rho_n^{(t)}$ is its remaining value under Eq. (1). Writing $S_t \subseteq E_t := \{(m, n) : m \in F, n \in Q_m, (m, n) \text{ feasible at } t\}$ for the pairs assigned in the stage, the value destroyed in the stage is

$$f_t(S_t) = \sum_{n=1}^N V_n \rho_n^{(t)} \left(1 - \prod_{m:(m,n) \in S_t} (1 - P_{m,n}) \right). \quad (19)$$

The objective f_t is *monotone submodular* [58]: adding a pair to S_t never decreases f_t (monotonicity), and its marginal gain never increases as S_t grows (submodularity), which here means that an additional weapon on a target helps less when other weapons are already assigned to it. The feasibility constraint is a *partition matroid* over E_t : the pairs are partitioned by weapon, and an assignment is feasible if it contains at most one pair from each weapon's block. Maximizing a monotone submodular function under a matroid constraint is NP-hard [18], so no polynomial-time method, the auction included, solves the stage problem optimally. We therefore aim for an assignment that is not optimal but provably good.

The mechanism keeps the auction's structure, repeated highest-bid assignment, and changes only the bid. Let $S_n \subseteq F$ be the weapons already assigned to target n in this stage, and let

$$\sigma_n = \prod_{m' \in S_n} (1 - P_{m',n}) \quad (20)$$

be the survival factor accumulated at target n . The marginal value of assigning weapon m to target n is

$$b_{m,n} = V_n \rho_n^{(t)} \sigma_n P_{m,n}. \quad (21)$$

The auction repeatedly selects the pair (m, n) with the largest $b_{m,n}$ over all unassigned $m \in F$ and legal n , records the assignment, and updates σ_n ; assignments are never undone. Because the bid of Eq. (21) equals the exact marginal gain of f_t , this procedure is the greedy algorithm for maximizing f_t under the partition matroid constraint, and its guarantee comes from submodular maximization theory rather than from auction duality.

Proposition 1 (The many-to-one auction is matroid greedy on the stage objective) *Fix stage t and a firing set F , and let $OPT_t(F)$ be the maximum of f_t over feasible allocations of the weapons in F . The many-to-one auction returns an allocation S_t^{auc} with*

$$f_t(S_t^{auc}) \geq \frac{1}{2} OPT_t(F). \quad (22)$$

Proof First, the auction's bid equals the exact marginal gain of f_t . Let S be the allocation so far and $S_{n'}$ the weapons already assigned to target n' . Adding (m', n') changes only the n' summand of Eq. (19):

$$\begin{aligned} f_t(S \cup \{(m', n')\}) - f_t(S) &= V_{n'} \rho_{n'}^{(t)} \left[\prod_{m \in S_{n'}} (1 - P_{m,n'}) \right. \\ &\quad \left. - \prod_{m \in S_{n'} \cup \{m'\}} (1 - P_{m,n'}) \right] \\ &= V_{n'} \rho_{n'}^{(t)} \left(\prod_{m \in S_{n'}} (1 - P_{m,n'}) \right) [1 - (1 - P_{m',n'})] \\ &= V_{n'} \rho_{n'}^{(t)} \sigma_{n'} P_{m',n'} = b_{m',n'}, \end{aligned}$$

which is the quantity of Eq. (21). Second, the auction is greedy in that quantity: each iteration takes the single largest bid over all unassigned weapons and legal targets, and never revokes an assignment. It is therefore the greedy algorithm for maximizing the monotone submodular f_t under the partition matroid constraint, which attains at least $\frac{1}{2}$ of the constrained optimum [20].

The factor $\frac{1}{2}$ is a worst-case floor that holds regardless of the scale of the problem: it does not degrade as the number of weapons M grows. It is also close to the best attainable, since no polynomial-time method can guarantee more than $1 - 1/e$ for this problem class [18], and the alternatives analyzed in Section 6.5 carry either no guarantee or one that decays as $1/M$. The floor is also loose in practice. On the $(5, 5, 5)$ and $(5, 7, 5)$ benchmark configurations, where $\text{OPT}_t(F)$ can be computed exactly by enumerating every feasible allocation of the firing weapons, the auction attains a mean 99.3% of the stage optimum over 77 policy-generated stages (ten instances per configuration, distribution of Section 7.1) and never falls below 87.1%. The $\frac{1}{2}$ bound is thus a guarantee against adversarial instances, not a description of typical behavior.

The auction optimizes a single stage, and that is exactly its limitation. Holding fire has zero value within the stage; it pays off only across stages, when ammunition saved now meets a more valuable target later. A per-stage optimizer therefore never holds, no matter how good its assignments are, and used alone the auction is weak (Section 7.4). This multi-stage decision, fire or hold, is what the learned policy of Section 6 provides, and the auction decides the rest.

6 Proposed Method: SCoPE

Most reinforcement learning approaches to (D)WTA train a policy that directly outputs which weapon engages which target, or skips an engagement in favor of a more effective target in the future [46, 21, 61]. That is, the trained policy is responsible for two different tasks at once: it must judge whether each weapon should fire at all, and it must choose the best target among the candidates.

We separate these two tasks. The policy aims to learn only the strategic decisions, fire or hold, and the target assignments are made with the auction algorithm of Section 5. The reason is that the two tasks have different structures. Once the firing weapons are known, the assignment is determined entirely by information available within the stage, such as kill probabilities and remaining target values, and the auction solves it with a provable quality guarantee (Proposition 1). The fire-or-hold decision cannot be solved within the stage, because the value of holding appears only in later stages; this is the part that must be learned.

We propose SCoPE, a reinforcement learning framework for DWTA. The state is encoded as a bipartite heterogeneous weapon-target graph with a graph neural network, and a weapon coordination layer is placed between the encoder and the decoder. The decoder makes the strategic decisions with a *parallel multi-pointer* policy: at every time step, all weapons with available ammunition choose fire or hold simultaneously, rather than one weapon deciding at a time. The auction algorithm then assigns targets to the weapons that decided to fire. Finally, one more learned policy acts as a reviewer of the completed schedule: it rectifies individual fire-or-hold decisions, re-simulates, and keeps the best result. Algorithm 1 states the full procedure, and Fig. 1 illustrates it.

6.1 Heterogeneous Graph Encoding

Each weapon-target pair state can be represented as a vector in $\mathbb{R}^9$ (Eq. (16)). We encode this state with a graph neural network rather than feeding the $M \times N$ vectors to the policy directly, for two reasons. First, only the damage rate $P_{m,n}$ is a property of the pair; the remaining features belong to a weapon or to a target alone, and a graph stores each of them once instead of copying them across all pairs. Second, a graph network has no dimension tied to M or N , so one trained model applies to any problem size, which is the property the zero-shot generalization in Section 7 rests on.

The graph is bipartite: weapon nodes $\mathbf{x}_m^{\text{wpn}} \in \mathbb{R}^4$, target nodes $\mathbf{x}_n^{\text{tgt}} \in \mathbb{R}^4$, and one edge carrying $\mathbf{x}_{m,n}^{\text{edge}} = P_{m,n}$ for each compatible pair $n \in Q_m$, so engagement compatibility is part of the graph topology. Each node and edge is first projected to a common hidden width H :

$$\mathbf{h}_m^{\text{wpn},(0)} = \mathbf{W}^{\text{wpn}} \mathbf{x}_m^{\text{wpn}} + \mathbf{b}^{\text{wpn}}, \quad \mathbf{h}_n^{\text{tgt},(0)} = \mathbf{W}^{\text{tgt}} \mathbf{x}_n^{\text{tgt}} + \mathbf{b}^{\text{tgt}}, \quad \mathbf{h}_{m,n}^{\text{edge},(0)} = \mathbf{W}^{\text{edge}} \mathbf{x}_{m,n}^{\text{edge}} + \mathbf{b}^{\text{edge}}. \quad (23)$$

Then L rounds of message passing follow [22]. At round ℓ , each edge is updated from its two endpoint embeddings and its own previous value,

$$\mathbf{h}_{m,n}^{\text{edge},(\ell)} = \tanh\left(\mathbf{W}_e \left[\mathbf{h}_m^{\text{wpn},(\ell-1)} \parallel \mathbf{h}_n^{\text{tgt},(\ell-1)} \parallel \mathbf{h}_{m,n}^{\text{edge},(\ell-1)}\right]\right), \quad (24)$$

and each weapon node aggregates the messages of its incident edges through a GRU,

$$\mathbf{h}_m^{\text{wpn},(\ell)} = \text{GRU}\left(\sum_{n \in Q_m} \mathbf{W}_w \left[\mathbf{h}_m^{\text{wpn},(\ell-1)} \parallel \mathbf{h}_n^{\text{tgt},(\ell-1)} \parallel \mathbf{h}_{m,n}^{\text{edge},(\ell)}\right], \mathbf{h}_m^{\text{wpn},(\ell-1)}\right), \quad (25)$$

with the target update $\mathbf{h}_n^{\text{tgt},(\ell)}$ defined analogously, aggregating over the weapons with $n \in Q_m$; $\parallel$ denotes concatenation. After L rounds, each weapon embedding $\mathbf{h}_m^{\text{wpn},(L)}$ reflects the state of the whole battlefield, and each edge embedding $\mathbf{h}_{m,n}^{\text{edge},(L)}$ carries the pair information that the decoder scores (Section 6.3).

6.2 Weapon-to-Weapon Communication

The graph encoding gives every weapon the same view of the battlefield, but no view of the other weapons' current decisions. Weapons in similar situations therefore make similar decisions: if the shared state favors firing, many weapons fire, whether or not the stage needs that many shots. The auction cannot correct this, because it only assigns targets among weapons that have already decided to fire. Under resource abundance this error is minor. Under scarcity, how many weapons should fire in a given stage is the central decision, and nothing in the encoder addresses it.

We address it with one self-attention layer over the M weapon embeddings, applied between the encoder and the scoring heads:

$$\mathbf{H}^{\text{wpn}} = \left[\mathbf{h}_1^{\text{wpn},(L)}; \dots; \mathbf{h}_M^{\text{wpn},(L)}\right] \in \mathbb{R}^{M \times H}, \quad \mathbf{C} = \text{MHA}(\mathbf{H}^{\text{wpn}}, \mathbf{H}^{\text{wpn}}, \mathbf{H}^{\text{wpn}}) \quad (26)$$

$$\mathbf{h}_m^{\text{wpn,comm}} = \text{Res}\left(\text{LN}\left(\mathbf{h}_m^{\text{wpn},(L)} + \mathbf{C}_m\right)\right) \quad (27)$$

where MHA is standard multi-head self-attention [55] over the weapon axis. Unlike a mean-pooled context vector, attention lets each weapon weight the specific weapons relevant to its own decision, for example those competing for the same compatible targets or those still holding ammunition. Both scoring heads in Section 6.3 consume $\mathbf{h}_m^{\text{wpn,comm}}$ (Eqs. (28)–(29)). The added cost is one attention pass over M tokens per stage, independent of N , so the decoding-call complexity of the parallel decoder is unchanged.

We call the architecture without this layer SCoPE and with it SCoPE-Comm, and report both. Comparing the two isolates the value of coordination over the participation decision, since coordination over target choice is already handled by the auction (Section 7.5). The released implementation additionally carries a learned scalar that can mix a Sinkhorn-normalized bonus into the edge scores; it is initialized at zero, trains to 0.021, and forcing it to zero at inference changes the mean objective by less than 10^{-4} across the twelve held-out configurations, so the term is inactive and plays no role in the results reported here.

6.3 Parallel Multi-Pointer Policy

Unlike a single global policy over all $M \times N$ (weapon, target) pairs, as in earlier neural WTAP formulations [46], SCoPE decodes M independent distributions simultaneously, one per weapon, each over that weapon’s compatible targets plus a no-op. Both scoring heads read the communicated weapon embedding of Eq. (27):

$$\text{score}_{m,n} = \mathbf{w}_e^\top \text{Res}\left(\mathbf{W}_c \left[\mathbf{h}_{m,n}^{\text{edge},(L)} \parallel \mathbf{h}_m^{\text{wpn,comm}} \right]\right), \quad n \in Q_m \quad (28)$$

$$\text{score}_{m,\text{noop}} = \mathbf{w}_0^\top \text{Res}\left(\mathbf{W}_0 \left[\mathbf{h}_m^{\text{wpn,comm}} \parallel \bar{\mathbf{h}}^{\text{wpn}} \parallel \bar{\mathbf{h}}^{\text{tgt}} \right]\right) \quad (29)$$

where $\text{Res}(\mathbf{x}) = \mathbf{x} + \mathbf{W}_2^{\text{res}} \sigma(\mathbf{W}_1^{\text{res}} \text{LN}(\mathbf{x}))$ is a residual block with layer normalization, $\mathbf{W}_c$ and $\mathbf{W}_0$ project their concatenated inputs to $\mathbb{R}^H$, and $\bar{\mathbf{h}}^{\text{wpn}} = \frac{1}{M} \sum_m \mathbf{h}_m^{\text{wpn,comm}}$ and $\bar{\mathbf{h}}^{\text{tgt}} = \frac{1}{N} \sum_n \mathbf{h}_n^{\text{tgt},(L)}$ are pooled global-context vectors. The per-weapon policy is

$$\pi_m(\cdot \mid s_t) = \text{softmax}\left([\text{score}_{m,n}]_{n \in Q_m}, \text{score}_{m,\text{noop}}\right), \quad (30)$$

masked to the targets currently legal under the constraints of Section 3 (reload, ammunition, time window). The no-op is always legal, and a weapon that is reloading or out of ammunition is deterministically assigned the no-op and contributes no gradient term in that stage. All M policies are computed in one forward pass and sampled independently, giving the joint action $\mathbf{a}_t = (a_{1,t}, \dots, a_{M,t})$ with

$$\log \pi_\theta(\mathbf{a}_t \mid s_t) = \sum_{m=1}^M \log \pi_m(a_{m,t} \mid s_t). \quad (31)$$

The per-stage cost is one network call, compared with M calls for a sequential decoder.

At inference, only the strategic half of each policy’s output is kept:

$$\text{fire}_m := \mathbb{K} \left[\arg \max_n \pi_m(n \mid s_t) \neq \text{noop} \right], \quad (32)$$

and the auction of Section 5 assigns targets over the firing set $F = \{m : \text{fire}_m = 1\}$. The policy's own target choice is discarded, since Proposition 1 shows the auction handles the assignment with a guarantee no learned rule matches; Section 7.4 confirms empirically that it does not.

Because the M policies are sampled independently, several weapons may select the same target in the same stage. This is a feasible action under the constraints of Section 3, so we do not prevent it with a conflict handler as in hard-coupled problems [3]. Independent sampling instead introduces two kinds of coordination error. The first, multiple weapons converging on the same target, concerns assignment and is corrected by the auction. The second, the wrong number of weapons deciding to fire at all, concerns participation, and the auction inherits it unchanged. The communication layer of Section 6.2 exists for the second error, and Section 7.5 measures both.

6.4 Learned Repair of the Completed Schedule

Every mechanism above decides within a stage and then moves on. Whether a stage- t decision was good often becomes observable only later: whether the ammunition it spent turned out to be needed afterwards, or whether the target it left alive was still available when its window closed. A construction policy, parallel or sequential, cannot revisit a stage once it has passed.

We therefore add a second learned policy that operates on the completed schedule, and again it edits only the strategic decision. Let $\phi \in \{0, 1\}^{T \times M}$ be a flip mask over stage-weapon slots. The repaired participation decision at slot (t, m) is $\text{fire}_m^{(t)} \oplus \phi_{t,m}$: where $\phi_{t,m} = 1$, a weapon that fired instead holds, and a weapon that held instead fires. The episode is re-simulated from the start under the modified decisions, with the auction reassigning targets as before. The repair policy scores all $T \times M$ slots from the states recorded along the current trajectory, flips the highest-scoring unflipped slot, re-simulates, and repeats for a budget of K edits, keeping the best schedule found.

Two properties follow from this construction, independent of training.

Proposition 2 (Monotone improvement and budget monotonicity) *Let J_K denote the objective returned after a budget of K edits under the keep-best rule. Then $J_K \leq J_0$ for every $K \geq 0$, and $J_{K'} \leq J_K$ whenever $K' \geq K$.*

Proof The returned schedule is the best of those evaluated, and the unedited schedule is always among them, so $J_K \leq J_0$. Edits are applied greedily and never undone, so the schedules evaluated under budget K are a prefix of those evaluated under $K' \geq K$; the minimum over a superset cannot increase.

The first inequality makes the layer safe to deploy: a repair policy that has learned nothing useful cannot degrade the schedule it is given, because its edits are simply discarded. The second makes K an anytime control: the same trained policies can run with small K when the decision cycle is tight and large K when it is not, without retraining.

Feasibility requires no special handling. A flip changes only the fire-or-hold intent at one slot. The resulting assignment is still computed by the auction against the environment's legality mask (readiness, ammunition, time window, Q_m), and the

episode is re-simulated from the start rather than patched in place, so all downstream consequences are recomputed exactly. An edit that asks a weapon to fire when it has no legal target is absorbed and leaves the schedule unchanged; its only cost is one unit of budget. Candidate slots are therefore restricted to those that currently fire or are currently legal.

The repair policy is trained by REINFORCE over the edit sequence, with the reward for each edit being the objective improvement that edit produced (Section 6.6). The edit operator is the simplest one that can express both directions of the participation decision. We also implemented a move operator that reschedules a weapon's shot from one stage to another, and a combined operator that can choose either type of edit at each step. Under an identical frozen-construction protocol in earlier experimentation, the three operators finished within a fraction of a percentage point of one another, with no operator clearly dominating; we kept the single flip for its simplicity and for the smaller action space it gives the jointly trained repair policy. That comparison predates the many-to-one pipeline and has not been re-run under it (Section 7.6).

We denote the full pipeline, SCoPE-Comm followed by the repair layer, as SCoPE-Repair.

6.5 Theoretical Analysis

Proposition 1 established what the auction guarantees. This section places the two decoding structures against the same yardstick, and records the structural defect of the one-to-one auction that motivated the many-to-one variant. Throughout, f_t , E_t , and the partition matroid are as in Section 5, with $F = \{1, \dots, M\}$, and $\text{OPT}_t = \max_{S_t \text{ feasible}} f_t(S_t)$.

Proposition 3 (Sequential decoding carries a constant-factor guarantee)

An idealized sequential decision rule, in which each weapon in turn selects the feasible pair of largest marginal gain given all pairs already selected in the stage, is the greedy algorithm for monotone submodular maximization under a matroid constraint, and therefore guarantees $f_t(\text{Seq}_t) \geq \frac{1}{2} \text{OPT}_t$ [20].

Proposition 4 (Parallel decoding without communication does not: tight

$\Theta(1/M)$) *Consider the idealized zero-communication parallel rule: each weapon m simultaneously selects its own feasible pair of largest singleton value $f_t(\{(m, n)\})$, with no visibility into the other weapons' same-stage choices. Then*

$$f_t(\text{Par}_t) \geq \frac{1}{M} \text{OPT}_t, \quad (33)$$

and this is tight: there are instances on which $f_t(\text{Par}_t)/\text{OPT}_t \rightarrow 1/M$.

Proof Lower bound. Let $e^* = \arg \max_{e \in E_t} f_t(\{e\})$. By monotonicity and submodularity, f_t is subadditive, so for the optimal set S_t^* (with $|S_t^*| \leq M$): $\text{OPT}_t = f_t(S_t^*) \leq \sum_{e \in S_t^*} f_t(\{e\}) \leq M f_t(\{e^*\})$. The weapon owning e^* selects it under the rule above, since e^* is globally best and hence best within its own part, so by monotonicity $f_t(\text{Par}_t) \geq f_t(\{e^*\}) \geq \text{OPT}_t/M$.

Tightness. Take M weapons and $M+1$ targets: one shared target n_0 with value 1, and one private target per weapon with value $1 - \varepsilon$, all with $P \equiv 1$ and all pairs

feasible. Every weapon's best singleton is n_0 , so Par_t sends all M weapons to n_0 and $f_t(\text{Par}_t) = 1$, while assigning one weapon to n_0 and the rest to their private targets gives $\text{OPT}_t = 1 + (M-1)(1-\varepsilon)$. The ratio tends to $1/M$ as $\varepsilon \rightarrow 0$.

The comparison explains the pipeline. Sequential decoding pays M network calls per stage for a $\frac{1}{2}$ guarantee. Zero-communication parallel decoding pays one call but can lose a factor of M . SCoPE's combination, parallel fire-or-hold decisions followed by the auction, recovers the $\frac{1}{2}$ guarantee for the assignment (Proposition 1) at one network call plus one auction per stage; what it does not recover by any guarantee is the participation decision, which is exactly the part that is learned.

The remaining two results explain why the classical one-to-one auction, applied to the stage problem as-is, is structurally inadequate, which is what motivated changing the bid rather than borrowing the algorithm unchanged.

Lemma 1 (The eviction auction enforces one-to-one assignment) *In the classical auction, an agent claiming an object evicts its current holder, so in any stage the procedure assigns at most one weapon to each target.*

Proof The auction maintains a map from targets to their currently assigned weapon. When weapon m claims target n , any incumbent holding n is evicted and returned to the unassigned queue before m is recorded as n 's owner, so the map remains single-valued after every bid. It is single-valued initially and the procedure terminates with it single-valued, so no target ends the stage with two assigned weapons.

Corollary 1 (Excluded optima) *If the stage-optimal allocation assigns two or more weapons to some target, the eviction auction cannot produce it, and such instances exist.*

Proof The first claim is immediate from Lemma 1. For existence, take two weapons and two targets with $P \equiv \frac{1}{2}$, remaining values $V_1\rho_1 = 1$ and $V_2\rho_2 = \varepsilon$, and all pairs feasible. Concentrating both weapons on target 1 destroys $1 - (\frac{1}{2})^2 = \frac{3}{4}$; spreading them destroys $\frac{1}{2} + \frac{\varepsilon}{2}$, which is smaller for $\varepsilon < \frac{1}{2}$. The optimum therefore assigns both weapons to target 1, which Lemma 1 forbids.

The loss identified by Corollary 1 is invisible in the objective values the mechanism reports, because the excluded allocations are never proposed. Detecting it requires examining the structure of the assignment rather than its value: the ratio of distinct engaged targets to firing weapons equals exactly one under eviction by Lemma 1, while the many-to-one auction lets it fall below one whenever concentrating fire pays.

Remark 1 Propositions 3 and 4 concern idealized decision rules, not our trained networks, and they bound single stages rather than full T -stage trajectories. The measured gap between the trained parallel and sequential decoders is far smaller than the worst case (Section 7.5), for three structural reasons. First, the worst case of Proposition 4 requires many weapons' top choices to coincide while near-equal alternatives go untouched, but SCoPE's scores are all conditioned on the same shared state, including each target's remaining value, which suppresses uninformed pile-on. Second, diminishing returns caps the loss of each collision: the foregone value of a k -way collision is at most the sum of the colliding weapons' next-best marginals. Third, the bound is per-stage while the process is multi-stage: value missed in one stage remains on the battlefield, is re-observed through the updated $\rho_n^{(t+1)}$, and can be recovered later.

6.6 Training: Reward-to-Go REINFORCE

The construction policy and the repair policy are trained with the same estimator. Neither uses a critic.

Reward. The per-step reward is Eq. (18), the fraction of initial target value destroyed at that step. One terminal term is added. The diagnosis in Section 7.5 showed that the trained policy often ended episodes with usable ammunition unspent, so the final step of each episode receives a bonus proportional to the fraction of available ammunition actually fired:

$$r_{T-1} \leftarrow r_{T-1} + \lambda \frac{\text{rounds fired}}{\text{rounds available}}, \quad \lambda = 0.1. \quad (34)$$

All neural methods in Section 7.4, including the sequential baseline, are trained with this term.

Credit assignment. We use reward-to-go: step t 's credit reflects only rewards realized at or after t ,

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}. \quad (35)$$

This return is normalized per instance across the P parallel rollouts, in the manner of the POMO shared baseline [33]:

$$\hat{A}_t = \frac{G_t - \frac{1}{P} \sum_{p=1}^P G_t^{(p)}}{\text{std}_p(G_t^{(p)})}, \quad (36)$$

and the actor is updated by

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad (37)$$

with $\log \pi_{\theta}(a_t | s_t)$ the joint per-weapon log-probability of Section 6.3. The factorization across weapons introduces no bias, because a product of per-weapon categorical distributions is itself a valid distribution over the joint action, so the score-function identity holds as usual. Eq. (36) departs from the unbiased estimator in two standard ways: the pooled mean includes the rollout being scored, an $O(1/P)$ bias common in shared-baseline practice [33], and the division by the standard deviation rescales stages relative to one another. The normalization coincides with the group-relative advantage of GRPO [50], but without importance sampling, clipping, or a KL penalty.

No critic. Our first design used a learned critic $v(s_t)$ as a potential function for reward shaping [47], to give the no-op action a same-step learning signal, since its immediate reward is always zero and its credit otherwise arrives only through later rewards. Potential-based shaping is policy-invariant, so the design was safe in principle. In practice it hurt: removing the critic and shaping outperformed the full mechanism at every problem size tested in earlier experimentation (not yet re-run under the current benchmark; Section 7.7). A likely reason is that G_t is already an exact Monte Carlo estimate of the return-to-go, and a jointly-trained critic replaces part of it with an approximation. The final recipe uses no critic.

Repair policy. The repair policy is trained with the same estimator on its own episode, the sequence of edits to one schedule (two edits per training step). Its reward per edit is the objective change that edit produced under re-simulation. Credit assignment is easier than for construction, since each edit’s reward measures that edit alone rather than a return shared across the whole episode. Edits are sampled during training and chosen by argmax at inference. The two policies are trained jointly on the same instance stream, both updated at every training step, so the repair policy learns on the schedules the construction policy actually produces rather than on those of a frozen proposer; the repair policy uses its own learning rate (Table 5).

Algorithm 1 SCoPE inference: construction followed by learned repair

Require: instance (V, P, Q, A, D, TW) ; construction policy π_θ ; repair policy ψ_ω ; edit budget K

Ensure: feasible schedule x

– **Pass 1: construction** –

- 1: $s_1 \leftarrow$ initial state
- 2: **for** $t = 1$ **to** T **do**
- 3: $H^{\text{wpn}}, H^{\text{tgt}}, H^{\text{edge}} \leftarrow \text{ENCODE}(s_t)$ ▷ L message-passing rounds, Sec. 6.1
- 4: $H^{\text{wpn}} \leftarrow \text{COMMUNICATE}(H^{\text{wpn}})$ ▷ self-attention over weapons, Sec. 6.2
- 5: $\pi_1, \dots, \pi_M \leftarrow \text{SCORE}(H^{\text{wpn}}, H^{\text{edge}}, \text{legal}(s_t))$ ▷ one forward pass, Sec. 6.3
- 6: $\text{fire}_m \leftarrow \mathbb{I}[\arg \max \pi_m \neq \text{noop}]$ for all m
- 7: $a_t \leftarrow \text{AUCTION}(\{m : \text{fire}_m\}, s_t)$ ▷ many-to-one, Sec. 5
- 8: $s_{t+1} \leftarrow \text{STEP}(s_t, a_t)$ ▷ apply hits, ammunition, reload, windows
- 9: **end for**
- 10: $x^* \leftarrow x \leftarrow$ schedule from $a_{1:T}$; $\phi \leftarrow \mathbf{0}$
- **Pass 2: repair (Sec. 6.4)** –
- 11: **for** $k = 1$ **to** K **do**
- 12: $z \leftarrow \psi_\omega(\text{states of } x)$ ▷ score all $T \times M$ slots
- 13: $(t^*, m^*) \leftarrow \arg \max \{z_{t,m} : \phi_{t,m} = 0, (t, m) \text{ eligible}\}$
- 14: $\phi_{t^*, m^*} \leftarrow 1$
- 15: $x \leftarrow \text{RESIMULATE}(\text{fire} \oplus \phi)$ ▷ feasibility recomputed, not patched
- 16: **if** $J(x) < J(x^*)$ **then** $x^* \leftarrow x$
- 17: **end if**
- 18: **end for**
- 19: **return** x^*

7 Experimental Results

7.1 Training and Test Data

SCoPE was trained with a multi-scale curriculum (random (M, N, T) drawn each episode, $M, N, T \in [5, 7]$) under two instance distributions, shown in Table 3. The *abundant* distribution follows the parameter ranges conventionally used to benchmark WTA solvers – fixed ammunition drawn independently of the time horizon, target values and time windows uncorrelated. The *scarce* distribution ties ammunition to the time horizon ($A_m \sim \mathcal{Z}(1, \lfloor T/2 \rfloor + 1)$, so no weapon can engage every stage) and correlates target value with emergence time ($V_n = V_{\text{base}} + V_{\text{bonus}} \cdot t_{n,\text{start}} / (T - 1)$, with $V_{\text{base}} \sim U(2, 6)$ and $V_{\text{bonus}} \sim U(2, 5)$ redrawn per target): later-emerging targets tend to be somewhat more valuable, with substantial per-target noise rather

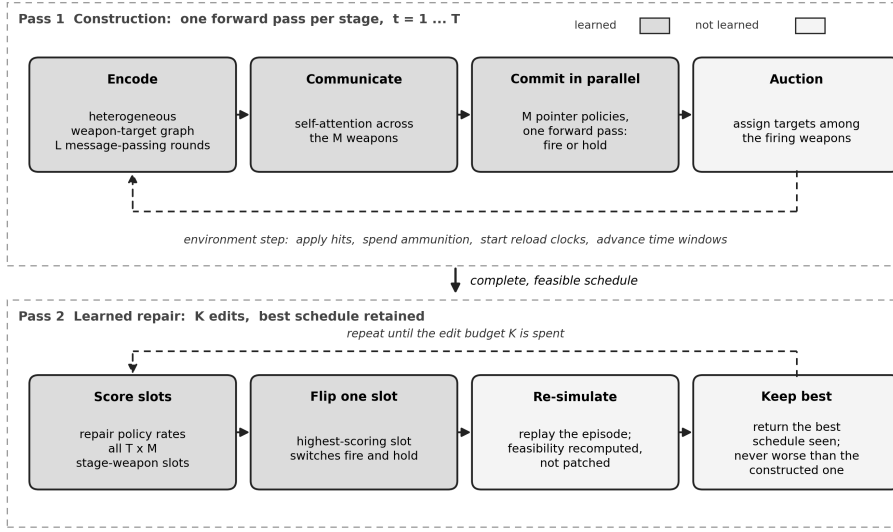


Fig. 1 The SCoPE pipeline, corresponding step for step to Algorithm 1. Construction repeats the encode, communicate, decide, and auction steps once per stage, with the environment transition closing the loop. The completed schedule then passes to the repair layer, which scores every stage-weapon slot, flips one, re-simulates, and keeps the best result over K edits. Shading distinguishes learned components from fixed ones.

than a deterministic split, so whether to hold a weapon in reserve for one is a genuine, instance-dependent judgment rather than a scripted pattern. As argued in Section 1, the scarce distribution is the one we treat as the paper’s primary evaluation condition; the abundant distribution is reported alongside it specifically to show where the simpler, communication-free hypothesis holds and where it does not (Section 7.5). Testing was performed on 12 held-out configurations spanning four size tiers – Small, Medium, Large, and Battlefield – shown in Table 4, drawn from the same distribution used in training (abundant or scarce) but at scales the policy never saw during training. Configurations are restricted to $N \geq M$ (targets/threats at least as numerous as own weapons), reflecting the resource-constrained defense scenario this work targets rather than an arbitrary grid.

Table 3 Parameter Distributions. $\mathcal{Z}(a, b)$ denotes a discrete uniform distribution on $\{a, \dots, b\}$.

Attribute	Abundant	Scarce (primary)
Target Value V_n	$U(1, 10)$	$U(2, 6) + U(2, 5) \cdot \frac{t_{n, \text{start}}}{T-1}$
Reload Time D_m	$\mathcal{Z}(1, 2)$	$\mathcal{Z}(0, 1)$
Ammunition A_m	$\mathcal{Z}(3, 5)$	$\mathcal{Z}(1, \lfloor T/2 \rfloor + 1)$
Start Time $t_{n, \text{start}}$	$\mathcal{Z}(0, \lfloor T/2 \rfloor)$	$\mathcal{Z}(0, T-1)$
End Time $t_{n, \text{end}}$	$T-1$	$T-1$
Damage Rate P	$U(0.2, 0.7)$	$U(0.3, 0.9)$

Table 4 Test problem configurations. Each configuration uses ten fixed, seed-controlled held-out instances, identical across all evaluated methods.

Tier	(M,N,T)	Instances
Small	(5,5,5)	10
	(5,7,5)	10
	(10,15,5)	10
Medium	(15,15,5)	10
	(15,20,5)	10
	(20,30,5)	10
Large	(30,30,10)	10
	(30,40,10)	10
	(40,50,10)	10
Battlefield	(50,50,15)	10
	(50,70,15)	10
	(70,100,15)	10

7.2 Hyperparameters

Table 5 shows the model configuration.

Table 5 Hyperparameters (pulled from the current training configuration, not the earlier transformer-based design)

Parameter	Value
Message-passing layers L	3
Hidden/embedding dimension H	128
Communication attention heads (SCoPE-Comm, Eq. (26))	8
Dropout (attention and residual blocks)	0.1
Optimizer	Adam
Learning rate (construction)	5×10^{-4} (cosine decay to 1%)
Learning rate (repair)	1×10^{-4} (cosine decay to 1%)
Weight decay	1×10^{-2}
Gradient-norm clip	1.0
Instances per training step	15
Parallel rollouts per instance P	10
Repair edit budget K at evaluation	3 (sweep in Table 8)

7.3 Performance Measures and Baselines

Baselines are the exact solver SCIP (600s time limit per instance, best-found solution reported when the limit is reached without a proof of optimality); a classical greedy marginal-return heuristic that, each stage, assigns the single highest-marginal-value legal (weapon, target) pair repeatedly until none remains positive-valued; the auction refinement of Section 5 used in isolation (i.e. with the auction itself, rather than a learned policy, deciding fire-or-hold from immediate net value); the sequential single-agent decoder described in Section 6; and two standard neural combinatorial optimization construction policies adapted to DWTAP within the RL4CO framework [4] – the Attention Model (AM) [31] and POMO [33]. AM and POMO use the same heterogeneous compatibility structure (Q_m , $P_{m,n}$) as SCoPE, size-agnostic embeddings so a single trained policy is evaluated zero-shot across all size tiers exactly as SCoPE

is, and are trained under the same multi-scale curriculum and instance distributions (Table 3). All neural methods (Sequential, SCoPE, SCoPE-Comm, AM, POMO) are trained with REINFORCE and no critic, and, because the training objective is non-convex and (Section 7.5) empirically non-monotone in training progress for every method tested, we select each method’s evaluation checkpoint by best held-out performance on periodic checks during training rather than by final training step, and report that selection openly in Table 6 rather than the last-epoch weights. Performance is reported against the best-found solution among all baselines (Eq. (38)), rather than as a gap to claimed optimality, since SCIP does not prove optimality at every tested scale (Table 6).

7.4 Comparison with Baselines

$$\% \text{ Achieved} = \frac{\text{avg}(1 - \text{Obj}_{\text{model}})}{\text{avg}(1 - \text{Obj}_{\text{best-found}})} \times 100\% \quad (38)$$

Following the convention of prior WTA reinforcement learning work [21], we report performance as the ratio of *achieved* value (destruction) to the best-found solution’s achieved value, rather than a gap in *remaining* value. DWTA’s objective (Eq. (1)) is a normalized remaining-value fraction that can approach zero at the optimum, so a gap computed relative to remaining value divides by a small, instance-dependent denominator and can be highly sensitive as a result – a modest absolute difference in remaining value can appear as an extreme percentage when the best-found solution already destroys most of the target value. Eq. (38) avoids this by construction, since the achieved-value denominator is bounded away from zero whenever the best-found solution destroys a non-trivial share of target value.

Table 6 Main results on the *scarce* (primary) distribution, 12 held-out configurations: normalized remaining-value objective (lower is better), single seed, ten fixed test instances per configuration. SCIP entries note instances solved to proven optimality within the 600s limit; below Medium tier SCIP frequently fails to prove optimality (and at Battlefield scale fails to find a competitive incumbent at all), so it is not treated as ground truth there. Every neural method (AM, POMO, Sequential, SCoPE, SCoPE-Comm) is evaluated at its best held-out checkpoint (Section 7.4), not its final training step. % Achieved (Eq. (38)) is SCoPE-Comm against the best-found value among all seven other methods, per configuration.

Tier	Config	SCIP	Greedy	Auction	AM	POMO	Sequential	SCoPE	SCoPE-Comm	% Ach.
Small	(5,5,5)	0.148 (10/10)	0.598	0.362	0.249	0.243	0.169	0.260	0.183	95.9%
	(5,7,5)	0.199 (10/10)	0.623	0.455	0.446	0.447	0.307	0.353	0.318	85.2%
	(10,15,5)	0.192 (6/10)	0.619	0.418	0.446	0.488	0.300	0.326	0.277	89.5%
Medium	(15,15,5)	0.087 (0/10)	0.533	0.266	0.288	0.315	0.096	0.190	0.122	96.2%
	(15,20,5)	0.138 (0/10)	0.616	0.363	0.480	0.508	0.207	0.323	0.200	92.8%
	(20,30,5)	0.176 (1/10)	0.638	0.448	0.548	0.584	0.295	0.369	0.269	88.7%
Large	(30,30,10)	0.015 (0/10)	0.355	0.078	0.164	0.093	0.016	0.032	0.025	98.9%
	(30,40,10)	0.045 (0/10)	0.407	0.177	0.256	0.136	0.032	0.049	0.044	98.7%
	(40,50,10)	0.051 (0/10)	0.331	0.150	0.261	0.144	0.028	0.037	0.032	99.6%
Battlefield	(50,50,15)	0.157 (0/10)	0.361	0.156	0.195	0.077	0.088	0.115	0.096	97.9%
	(50,70,15)	0.992 (fail)	0.390	0.171	0.268	0.099	0.080	0.137	0.087	99.3%
	(70,100,15)	0.993 (fail)	0.403	0.177	0.306	0.106	0.100	0.151	0.105	99.4%

Mean % Achieved across all 12 configurations is 95.2%. SCoPE-Comm improves on plain SCoPE at every configuration. The best-found method is SCIP’s own (unproven, best-incumbent) solution at seven of the twelve configurations – Small tier fully, plus Medium and the smallest Large-tier configuration, where SCIP’s branch-and-bound still finds a strong incumbent even without proving it optimal – Sequential at four configurations (the remaining Large-tier and two of three Battlefield-tier

configurations, where SCIP either fails to prove optimality with a much larger gap or fails to find a competitive incumbent within the time limit at all), and POMO at one (50,50,15). % Achieved is $\geq 95\%$ at 8 of 12 configurations, all at Large and Battlefield tier plus (5,5,5) and (15,15,5); the four configurations below 95% – (5,7,5), (10,15,5), (15,20,5), (20,30,5) – are all Small or Medium tier, where SCIP still proves or nearly proves optimality (Table 6) and so leaves the least relative room for any method to be close to it. The pure auction baseline (Section 5’s mechanism with no learned participation decision) beats Greedy at every configuration, and even beats AM and POMO at (10,15,5), but trails Sequential, SCoPE, and SCoPE-Comm at every configuration, confirming that a within-stage assignment rule alone, however good, is not sufficient without a learned fire-or-hold judgment layered on top of it.

7.5 Coordination Analysis: Diagnosing and Closing the Participation Gap

This subsection substantiates the paper’s central empirical claim in two steps: first, an instance-level diagnostic that identifies *what* plain SCoPE gets wrong relative to exact-solver output; second, aggregate evidence for how much of that gap the communication layer of Section 6.2 closes, and how much is explained (or not) by the specific failure mode the diagnostic found.

Instance-level diagnosis. We compared SCoPE’s round-by-round decisions against SCIP’s proven-optimal schedule on individually inspected 5-weapon/5-target/5-stage instances from the scarce distribution. The qualitative pattern was consistent across every instance checked: SCIP’s optimal solution used every available weapon, while SCoPE’s decisions left a meaningful share of ammunition completely unused across the episode – not concentrated in one weapon, but a different weapon idle in each instance, consistent with a general under-commitment tendency rather than a fixed structural bias toward any particular weapon index. This motivated adding a terminal reward bonus proportional to the fraction of available ammunition actually expended (Section 6.6) to the training recipe used for every neural method reported in Table 6.

Aggregate ammo-utilization measurement. Having added that bonus to training, we then measured its effect directly: Table 7 reports the fraction of available ammunition actually fired, at each method’s best held-out checkpoint, across all 12 configurations of the scarce distribution. Both SCoPE and SCoPE-Comm expend essentially all available ammunition (99.8% and 98.4% mean utilization, respectively) – the under-utilization the instance-level diagnosis identified is, by this checkpoint, resolved for *both* variants, since both were trained with the same ammunition bonus. This is an important negative result for the mechanism story: it rules out “the communication layer causes more shots to be fired” as the explanation for its advantage in Table 6. If anything, SCoPE-Comm fires *slightly* less ammunition than plain SCoPE on average, while still achieving lower (better) remaining-value objectives at every configuration.

What this rules out matters more than what it confirms: whatever the communication layer adds, it is not simply “fire more.” The remaining, more likely explanation – that communication improves *when* each weapon fires and which target it is paired with by the auction, rather than *whether* it fires at all – is consistent with the mechanism described in Section 6.2 (the no-op score and the edge score both now depend on the communicated embedding, not only the no-op score as in plain SCoPE), but

Table 7 Ammunition-utilization rate (share of available ammunition actually fired), mean over 12 held-out configurations, scarce distribution, single seed, best checkpoint.

Method	Ammo utilization
SCoPE	99.8%
SCoPE-Comm	98.4%

we did not collect a metric in this revision that isolates timing/targeting quality independent of final destruction ratio. We report this as an open question (Section 8) rather than overstate a mechanism the data available here does not fully pin down.

7.6 Repair Layer: Contribution, Edit Budget, and Operator Choice

The repair layer is evaluated on the same twelve held-out configurations as Table 6 (scarce distribution, ten fixed instances per configuration, single seed), with edits selected greedily and the best schedule retained (Section 6.4). The construction policy here is the one trained jointly with the repair policy (Section 6.6); its construction-only quality is the $K=0$ column of Table 8 and differs slightly from the separately trained SCoPE-Comm column of Table 6.

Three questions are separable here and we report them separately. First, how much the layer contributes on top of the construction policy it repairs, which is the quantity that matters for the claim in the title. Second, how that contribution varies with the edit budget K : Proposition 2 guarantees the objective cannot increase with K , so the informative measurement is not whether the curve descends but where it stops descending – the budget past which further edits find nothing, which is also the point at which the layer has converged to a local optimum of its own neighbourhood. Third, whether the single-flip operator was the right choice against the richer alternatives described in Section 6.4.

Table 8 Repair edit-budget sweep: normalized remaining-value objective (lower is better) after K edits, scarce distribution, ten fixed instances per configuration, single seed. *flat@* is the mean edit index after which no further edit improved the schedule within the $K=100$ probe.

Config	$K=0$	$K=3$	$K=10$	$K=30$	$K=100$	flat@
(5,5,5)	0.2486	0.1929	0.1794	0.1794	0.1794	3.2
(5,7,5)	0.3213	0.2764	0.2696	0.2685	0.2685	3.9
(10,15,5)	0.2911	0.2686	0.2640	0.2640	0.2640	3.8
(15,15,5)	0.1215	0.1150	0.1108	0.1102	0.1102	5.0
(15,20,5)	0.2150	0.1949	0.1831	0.1831	0.1831	4.0
(20,30,5)	0.2599	0.2451	0.2302	0.2302	0.2302	5.7
(30,30,10)	0.0153	0.0133	0.0122	0.0120	0.0118	12.8
(30,40,10)	0.0361	0.0340	0.0331	0.0331	0.0331	3.8
(40,50,10)	0.0298	0.0293	0.0287	0.0287	0.0287	4.0
(50,50,15)	0.0972	0.0941	0.0917	0.0917	0.0917	7.1
(50,70,15)	0.0869	0.0842	0.0811	0.0809	0.0809	7.6
(70,100,15)	0.1103	0.1070	0.1063	0.1058	0.1057	14.9
Mean	0.1528	0.1379	0.1325	0.1323	0.1323	6.3

Table 8 answers the first two questions. On contribution: repair improves every one of the twelve configurations, and the mean objective falls from 0.1528 at $K=0$ to

0.1323 at convergence, a 13.4% relative improvement; the largest single-configuration gain is at (5,5,5), from 0.2486 to 0.1794 (27.8% relative). The repaired pipeline also improves on the mean of every construction-only method in Table 6, including Sequential (0.143): where it beats Sequential it does so clearly, at five configurations concentrated in the Small and Medium tiers, and where Sequential remains ahead, at Large and Battlefield tier, the margin is within a few thousandths of normalized objective, at a fraction of Sequential’s decoding cost.

On the budget: the curve is steep and short. $K=3$ already captures 73% of the total attainable improvement and $K=10$ captures 99%; past $K=30$ the objective is flat to four digits. Where the curve flattens moves with problem size, from a mean flat@ of roughly 3–4 edits at Small tier to 12.8 and 14.9 at the largest Large- and Battlefield-tier configurations, which is the behaviour the anytime-control reading of Proposition 2 predicts: a larger schedule offers more slots worth editing, and a larger budget keeps finding them, while a tight decision cycle can stop at $K=3$ and still bank most of the gain.

The third question, the operator comparison between single-flip, move, and combined edits under this protocol, has not yet been re-run under the current many-to-one pipeline and is reported qualitatively in Section 6.4 only.

7.7 Ablation: Critic and Potential-Based Shaping

Section 6.6 motivated SCoPE’s critic-free training recipe with the finding that a jointly-trained critic used as a potential function for reward shaping hurt held-out generalization on an earlier iteration of the simulator and instance distribution. That comparison has not yet been re-run under the current, scarce-distribution benchmark and simulator (Table 6 used no-critic training exclusively, so no head-to-head numbers exist to report here yet).

7.8 Inference Efficiency

Table 9 reports mean per-instance decode time on the same held-out benchmark (ten instances per configuration, two untimed warm-up instances, single GPU). The SCoPE-Comm column times the full construction pipeline of Algorithm 1, including the many-to-one auction at every stage; the Sequential column times the per-weapon decoding loop. SCoPE-Comm is 3.2 times faster at Small tier, and the advantage widens with scale to 5.6 times at Battlefield tier, consistent in direction with the $O(T)$ vs. $O(M \times T)$ decoding-call complexity argument of Section 6.3. The measured ratio is smaller than the call-count ratio M because the auction runs on the CPU inside the loop and its cost also grows with the number of firing weapons. At the largest configuration, $70 \times 100 \times 15$, the absolute difference is 4.84 versus 0.85 seconds per instance, which is the difference between one decision cycle and several at a battlefield re-planning tempo. The repair layer is not included in these times; each edit costs one additional re-simulation of the construction pass, so a budget of K multiplies the pipeline cost by at most $K+1$, and the anytime property of Proposition 2 lets the operator choose K to fit the available cycle.

Table 9 Mean per-instance decode time in seconds (tier averages over three configurations each, ten instances per configuration, single seed). SCoPE-Comm includes the many-to-one auction at every stage; repair is excluded (each edit adds one re-simulation).

Tier	Sequential	SCoPE-Comm	Speedup
Small	0.080	0.025	3.2×
Medium	0.229	0.047	4.9×
Large	1.148	0.222	5.2×
Battlefield	3.467	0.621	5.6×

8 Conclusion

This paper opened by asking when simultaneous, communication-free decoding is safe on DWTAP. The answer is conditional, and the condition is empirically load-bearing rather than a footnote: under resource-abundant instances, a parallel decoder with no learned coordination mechanism, refined only by a fixed marginal-value auction over already-firing weapons, is close to sufficient – consistent with the soft-coupling argument of Section 2, since contention there is merely unprofitable, not infeasible. Under resource-scarce instances – the regime we argue is the realistic one – it is not: the auction has no channel through which one weapon’s tentative commitment to fire can influence another’s, and that omission costs real performance once weapons and ammunition are genuinely limited (Table 6, plain SCoPE trailing Sequential by up to 36% relative objective at Medium scale). What restores near-parity is not a return to sequential decoding, but a single weapon-to-weapon self-attention layer added before the existing scoring heads (Section 6.2), closing the gap to a mean 95.2% of best-found value across twelve held-out scales – at the same decoding-call complexity order as the communication-free decoder.

For practitioners of multi-agent neural construction, two results transfer beyond DWTAP. First, a design test: before adding a conflict handler, ask whether the problem’s coupling is hard or soft, and separately, whether the resources being contended for are actually scarce in the instances you evaluate on – soft coupling under resource abundance can look like it needs no coordination machinery at all, a conclusion that resource scarcity alone can overturn without changing the coupling type. Second, a diagnostic discipline: our first hypothesis for why the communication layer helps – that it fixes systematic under-use of available ammunition, the specific failure an instance-level comparison against exact-solver output had identified – turned out to be incomplete once measured in aggregate (Section 7.5): both variants use essentially all their ammunition by the checkpoint reported, yet SCoPE-Comm still wins. The lesson is not that the diagnostic was wrong – it correctly motivated a training-recipe fix (the ammunition-utilization reward bonus, Section 6.6) that both variants now share – but that a mechanism confirmed at one training stage should not be assumed to still explain a gap measured at a later, better-trained one without re-checking.

Several limits bound these claims as reported here. Results throughout are single-seed; the closest calls in Table 6 (SCoPE-Comm against Sequential at Large and Battlefield tier, within 2 percentage points of normalized objective at several configurations) would benefit from a multi-seed variance estimate before the ranking is treated as settled. The critic-and-shaping ablation motivating Section 6.6’s training recipe predates this revision’s simulator fixes and has not yet been re-run under the current benchmark (Section 7.7); the same is true of the resource-abundant comparison invoked in the Introduction and at the top of this section, whose parallel-versus-

sequential parity numbers come from experiments before those fixes. We have not isolated a metric that directly confirms *why* the communication layer helps beyond ruling out ammunition under-use (Section 7.5); the timing/targeting explanation offered there is plausible and consistent with the mechanism’s design, but unconfirmed. And the soft-coupling test itself remains qualitative; we have not characterized where the hard/soft boundary lies for objectives that mix both elements.

These limits point at the questions we consider most worth answering. What specifically does the communicated embedding change about *when* a weapon fires and which target the auction pairs it with, beyond the aggregate destruction-ratio evidence reported here? Does the resource-scarcity distinction generalize to other soft-coupled multi-agent problems, or is it particular to WTA’s ammunition/reload structure? Can the hard/soft coupling boundary be made quantitative – for instance, through the curvature of the objective – so the design test of this section becomes a computable diagnostic rather than a qualitative judgment call? And how far does the repair layer’s operator set bound what it can recover? Single-flip edits reach only the participation decision, and richer operators performed no better than the flip in our earlier experimentation (Section 6.4); whether an operator that edits target assignments directly, or one that acts on a whole stage at once, would do better under a learned selector remains open.

Statements and Declarations

Competing Interests: The authors declare no competing interests.

Funding: No funding was received for this work.

Data Availability: The test instances, trained model checkpoints, and evaluation code are available at <https://github.com/jlee-105/BRELA>.

References

1. Ahuja, R.K., Kumar, A., Jha, K.C., Orlin, J.B.: Exact and heuristic algorithms for the weapon-target assignment problem. *Operations research* **55**(6), 1136–1146 (2007)
2. Bellman, R.: *Dynamic programming* (1957)
3. Berto, F., Hua, C., Luttmann, L., Son, J., Park, J., Ahn, K., Kwon, C., Xie, L., Park, J.: Parco: Learning parallel autoregressive policies for efficient multi-agent combinatorial optimization. In: *Advances in Neural Information Processing Systems* (2025). ArXiv:2409.03811
4. Berto, F., Hua, C., Park, J., Luttmann, L., Ma, Y., Bu, F., Wang, J., Ye, H., Kim, M., Choi, S., et al.: RL4co: an extensive reinforcement learning for combinatorial optimization benchmark. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (2025). ArXiv:2306.17100
5. Bertsekas, D.P.: The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research* **14**(1), 105–123 (1988)
6. Bertsekas, D.P., Castañon, D.A.: The auction algorithm for the transportation problem. *Annals of Operations Research* **20**(1), 67–96 (1989)
7. Bertsekas, D.P., Tsitsiklis, J.N., Wu, C.: Rollout algorithms for combinatorial optimization. *Journal of Heuristics* **3**(3), 245–262 (1997)

8. Bogrybayeva, A., Meraliyev, M., Mustakhov, T., Dauletbayev, B.: Machine learning to solve vehicle routing problems: A survey. *IEEE Transactions on Intelligent Transportation Systems* **25**(6), 4754–4772 (2024)
9. Bracken, J., McCormick, G.P.: Selected applications of nonlinear programming. Tech. rep., Research Analysis Corp Mclean va (1968)
10. Burkard, R., Dell’Amico, M., Martello, S.: *Assignment Problems*, revised reprint edn. SIAM, Philadelphia, PA (2012)
11. Chang, T., Kong, D., Hao, N., Xu, K., Yang, G.: Solving the dynamic weapon target assignment problem by an improved artificial bee colony algorithm with heuristic factor initialization. *Applied Soft Computing* **70**, 845–863 (2018)
12. Chen, X., Tian, Y.: Learning to perform local rewriting for combinatorial optimization. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
13. Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y.: Simulation-guided beam search for neural combinatorial optimization. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 8760–8772 (2022)
14. Das, A., Gervet, T., Romoff, J., Batra, D., Parikh, D., Rabbat, M., Pineau, J.: Tarmac: Targeted multi-agent communication. In: *Proceedings of the 36th International Conference on Machine Learning*, pp. 1538–1546 (2019)
15. Day, R.H.: Allocating weapons to target complexes by means of nonlinear programming. *Operations Research* **14**(6), 992–1013 (1966)
16. Delarue, A., Anderson, R., Tjandraatmadja, C.: Reinforcement learning with combinatorial actions: An application to vehicle routing. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 609–620 (2020)
17. Durgut, R., Kutucu, H., Akleyek, S.: An artificial bee colony algorithm for solving the weapon target assignment problem. In: *Proceedings of the 7th International Conference on Information Communication and Management*, pp. 28–31 (2017)
18. Feige, U.: A threshold of $\ln n$ for approximating set cover. *Journal of the ACM* **45**(4), 634–652 (1998)
19. Firstman, S.I.: An approximating algorithm for an optimum aim-points problem. *Naval Research Logistics Quarterly* **7**(2), 151–167 (1960)
20. Fisher, M.L., Nemhauser, G.L., Wolsey, L.A.: An analysis of approximations for maximizing submodular set functions ii. *Mathematical Programming Study* **8**, 73–87 (1978)
21. Gaudet, B., Drozd, K., Furfaro, R.: Deep reinforcement learning for weapons to targets assignment in a hypersonic strike. arXiv preprint arXiv:2310.18509 (2023)
22. Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: *International Conference on Machine Learning*, pp. 1263–1272 (2017)
23. Grinsztajn, N., Furelos-Blanco, D., Surana, S., Bonnet, C., Barrett, T.: Winner takes it all: Training performant rl populations for combinatorial optimization. In: *Advances in Neural Information Processing Systems*, vol. 36, pp. 48485–48509 (2023)
24. Hodson William T., I.: A linear programming approach to weapon allocation. Tech. rep., Air Force Institute of Technology (1971). Original bib entry had author/title fields transposed, producing the garbled in-text citation "M. W. T. H. Ill"; corrected here. Verify institution/report number against the original source before final submission.

25. Hottung, A., Kwon, Y.D., Tierney, K.: Efficient active search for combinatorial optimization problems. arXiv preprint arXiv:2106.05126 (2022)
26. Hu, T., Zhang, X., Luo, X., Chen, T.: Dynamic target assignment by unmanned surface vehicles based on reinforcement learning. *Mathematics* **12**(16), 2557 (2024)
27. Johansson, F., Falkman, G.: An empirical investigation of the static weapon-target allocation problem. In: *Proceedings of the 3rd Skövde Workshop on Information Fusion Topics*, pp. 63–67 (2009)
28. denBroeder Jr, G., Ellison, R., Emerling, L.: On optimum target assignments. *Operations Research* **7**(3), 322–326 (1959)
29. Kline, A., Ahner, D., Hill, R.: The weapon-target assignment problem. *Computers & Operations Research* **105**, 226–236 (2019). DOI 10.1016/j.cor.2018.10.015
30. Kolitz, S.E.: Analysis of a maximum marginal return assignment algorithm. In: *Proceedings of the 27th IEEE Conference on Decision and Control*, pp. 2431–2436. IEEE (1988)
31. Kool, W., Van Hoof, H., Welling, M.: Attention, learn to solve routing problems! arXiv preprint arXiv:1803.08475 (2018)
32. Kuhn, H.W.: The hungarian method for the assignment problem. *Naval Research Logistics Quarterly* **2**(1–2), 83–97 (1955)
33. Kwon, Y.D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S.: Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* **33**, 21188–21198 (2020)
34. Kwon, Y.D., Choo, J., Yoon, I., Park, M., Park, D., Gwon, Y.: Matrix encoding networks for neural combinatorial optimization. *Advances in Neural Information Processing Systems* **34**, 5138–5149 (2021)
35. Lee, Z.J., Lee, C.Y., Su, S.F.: An immunity-based ant colony optimization algorithm for solving weapon–target assignment problem. *Applied Soft Computing* **2**(1), 39–47 (2002)
36. Lee, Z.J., Su, S.F., Lee, C.Y.: Efficiently solving general weapon-target assignment problem by genetic algorithms with greedy eugenics. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **33**(1), 113–121 (2003)
37. Lee, Z.J., Su, S.F., Lee, C.Y.: A genetic algorithm for weapon-target assignment problems. *Applied Soft Computing* **3**(3), 263–275 (2003)
38. Lloyd, S.P., Witsenhausen, H.S.: Weapons allocation is np-complete. In: *1986 summer computer simulation conference*, pp. 1054–1058 (1986)
39. Luo, F., Lin, X., Liu, F., Zhang, Q., Wang, Z.: Neural combinatorial optimization with heavy decoder: Toward large scale generalization. *Advances in Neural Information Processing Systems* **36** (2023)
40. Luo, W., Lü, J., Liu, K., Chen, L.: Learning-based policy optimization for adversarial missile-target assignment. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **52**(7), 4426–4437 (2021)
41. Ma, Y., Li, J., Cao, Z., Song, W., Zhang, L., Chen, Z., Tang, J.: Learning to iteratively solve routing problems with dual-aspect collaborative transformer. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 11096–11107 (2021)
42. Manne, A.S.: A target-assignment problem. *Operations research* **6**(3), 346–351 (1958)
43. Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E.: Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*

- 134**, 105400 (2021)
44. Meng, F., Tian, K., Wu, C.: Deep reinforcement learning-based radar network target assignment. *IEEE Sensors Journal* **21**(14), 16315–16327 (2021)
45. Metler, W.A., Preston, F.L.: Suite of weapon assignment algorithms for a sdi mid-course battle manager. memorandum report. Tech. rep., Naval Research Lab., Washington, DC (USA) (1990)
46. Na, H., Ahn, J., Moon, I.C.: Weapon–target assignment by reinforcement learning with pointer network. *Journal of Aerospace Information Systems* **20**(1), 53–59 (2023)
47. Ng, A.Y., Harada, D., Russell, S.: Policy invariance under reward transformations: Theory and application to reward shaping. In: *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pp. 278–287 (1999)
48. Oh, S.H., Byeon, G.W., Cho, Y.I., Kwon, S., Woo, J.H.: Artificial intelligence in combat decision-making: Weapon target assignment via reinforcement learning and graph neural networks. *IEEE Transactions on Cybernetics* **56**(2), 631–643 (2026). DOI 10.1109/TCYB.2025.3610606
49. Park, J., Bakhtiyar, S., Park, J.: Schedulenet: Learn to solve multi-agent scheduling problems with reinforcement learning. *arXiv preprint arXiv:2106.03051* (2021)
50. Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y.K., Wu, Y., Guo, D.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024)
51. Shin, M.K., Park, S.S., Lee, D., Choi, H.L.: Mean field game based reinforcement learning for weapon-target assignment. *Journal of the Korea Institute of Military Science and Technology* **23**(4), 337–345 (2020)
52. Sonuc, E., Sen, B., Bayir, S.: A parallel simulated annealing algorithm for weapon-target assignment problem. *International Journal of Advanced Computer Science and Applications* **8**(4), 87–92 (2017)
53. Sukhbaatar, S., Fergus, R., et al.: Learning multiagent communication with back-propagation. In: *Advances in Neural Information Processing Systems*, vol. 29 (2016)
54. Sun, H., Guo, X., Bu, X., Wang, G.: A collaborative target assignment method for heterogeneous multi-firepower elements based on an adaptive grabbing-order mode. *Defence Technology* (2026). DOI 10.1016/j.dt.2026.07.002
55. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
56. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. *Advances in neural information processing systems* **28** (2015)
57. Wacholder, E.: A neural network-based optimization algorithm for the static weapon-target assignment problem. *ORSA Journal on computing* **1**(4), 232–246 (1989)
58. Wang, Z., Wang, X., Liang, Y., Pan, Q.: Weapon target assignment leveraging strong submodularity. In: *2013 IEEE International Conference on Information and Automation (ICIA)*, pp. 74–79. IEEE (2013)
59. Wu, Y., Lei, Y., Zhu, Z., Yang, X., Li, Q.: Dynamic multitarget assignment based on deep reinforcement learning. *IEEE Access* **10**, 75998–76007 (2022)
60. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and*

- Learning Systems **33**(9), 5057–5069 (2022). DOI 10.1109/TNNLS.2021.3068828
61. Yoon, C., Lee, J., Cho, J.: A transformer-based reinforcement learning approach for scalable weapon target assignment. *The Journal of Defense Modeling and Simulation* (2025). DOI 10.1177/15485129251335043
 62. Zhang, F., Hu, H., Zhao, Y.: Deep reinforcement learning with graph attention mechanism for vehicle routing problem with time windows. *Applied Intelligence* **55**(13), 937 (2025). DOI 10.1007/s10489-025-06829-z
 63. Zhang, K., Zhou, D., Yang, Z., Li, X., Zhao, Y., Kong, W.: A dynamic weapon target assignment based on receding horizon strategy by heuristic algorithm. In: *Journal of Physics: Conference Series*, vol. 1651, p. 012062. IOP Publishing (2020)
 64. Zhang, K., Zhou, D., Yang, Z., Zhao, Y., Kong, W.: Efficient decision approaches for asset-based dynamic weapon target assignment by a receding horizon and marginal return heuristic. *Electronics* **9**(9), 1511 (2020)